

乱数を用いたプログラム

釧路工業高等専門学校情報工学科5年・情報工学実験II

氏名：クリスティアン・ハルジュノ

出席番号：21

学籍番号：234071

提出日：2025年7月22日

目次

1	初めに	1
2	背景	1
2.1	5つのシミュレーションを紹介	1
2.1.1	課題1: 7つのサイコロを振る	1
2.1.2	課題2: 半径7の円の面積	2
2.1.3	課題3: 異身長男女を一行に並べる	3
2.1.4	課題4: ジャンケンの割合	5
2.1.5	課題5: ビンゴのシミュレーション	7
2.2	シミュレーションの役割	9
2.3	モンテカルロシミュレーション	10
3	実験方法	10
3.1	ハードウェアと環境	10
3.2	ランダム関数	11
3.3	フィッシャー-イエーツのシャッフル	13
3.4	モジュロによるランダム関数のバイアス	13
3.5	実行数	16
3.6	移動平均による値の収束	18
3.7	ジャンケンシミュレーションに用いる機械学習	18
4	実験結果	19
4.1	課題1: 7つのサイコロが同じ目を出すシミュレーション	19
4.2	課題2: 半径7の円の面積	21
4.3	課題3: 異身長男女を一行に並べる	24
4.4	課題4: ジャンケンの割合	25
4.4.1	相手が完全にランダムな場合 (バイアスなし)	26
4.4.2	相手の戦略が常に変化する場合 (機械学習)	27
4.5	課題5: ビンゴのシミュレーション	28
4.5.1	7回以内で当たる	29
4.5.2	最短回数 (4回)	29
4.5.3	当たりとなる最も遅いゲーム回数 (71回)	30
5	分析と理論	31
5.1	課題1: 7つのサイコロが同じ目を出すシミュレーション	31
5.2	課題2: 半径7の円の面積	32
5.3	課題3: 異身長男女を一行に並べる	33
5.4	課題4: ジャンケンの割合	33

5.5	課題 5: ビンゴのシミュレーション	34
6	まとめ	36
7	発表についての感想	37
7.1	齊藤 隼	37
7.2	神力 奏太	37
8	付録	38

1 初めに

統計の世界において、ある事象が起こる「確率」の計算は $\frac{\text{条件を満たす場合の数}}{\text{全体の事象数}}$ という単純な式で表される。たとえば、コインを投げて表が出る確率は $\frac{1}{2}$ であり、これはコインに 2 面が存在するためである。しかし、実際にコインを 10 回投げしてみると、表が出る割合は偏って見えることがあり、 $\frac{1}{3}$ や $\frac{1}{2}$ といった結果になる場合もある。ある意味で、確率には運による偏りの要素が依然として存在すると言える。このため、シミュレーションを多数回繰り返すことによって、「運」や「ノイズ」の影響を最小限に抑え、より正確な結果を得ることが重要である。

このレポートでは、乱数を用いたプログラムの実装とその結果の分析を行う。具体的には、コイン投げやサイコロの目の出方など、様々なランダムな事象をシミュレートし、それらの結果を統計的に解析することで、理論的な確率と実際の結果との関係を明らかにすることを目的とする。

2 背景

2.1 5つのシミュレーションを紹介

2.1.1 課題 1: 7つのサイコロを振る

本シミュレーションでは、7 個の 6 面サイコロを投げ、すべてのサイコロが同じ目を出す確率を求める。また、サイコロの個数を変化させながらシミュレーションを実行し、その結果を比較する。



図 1: 7つの同じ面のサイコロ

理論的には、一つの均質なサイコロにおいて各面が出る確率は正確に $\frac{1}{6}$ である。しかし、サイコロの個数を増やすことで、確率の分布は大きく変化する。 n 個のサイコロが同じ目を出す確率を計算するために、式 1 を用いる。

$$P = \frac{s}{s^n} = \frac{1}{s^{n-1}} \quad (1)$$

- n 個のサイコロのすべての組み合わせは s^n 通り。

- 全てのサイコロが同じ出目になる場合の数は s 通り（全て 1, 全て 2, ..., 全て s ）。
- よって, 確率は次のようになります：

$$P = \frac{\text{一致する場合の数}}{\text{全体の組み合わせ数}} = \frac{s}{s^n} = \frac{1}{s^{n-1}}$$

本実験では 7 個の 6 面サイコロをシミュレーションするため, すべてのサイコロが同じ目を示す確率は式 2 に示す通り, 2.143×10^{-5} となる。

$$P = \frac{1}{6^{7-1}} = \frac{1}{6^6} = \frac{1}{46656} \approx 2.143 \times 10^{-5} \quad (2)$$

2.1.2 課題 2: 半径 7 の円の面積

本シミュレーションでは, 2 次元平面上に半径 7 の円を描く。乱数により生成された位置 (x, y) のプロットを平面上に配置し, そのプロットが円の内部に存在する理論的確率を計算し, シミュレーション結果と比較する。

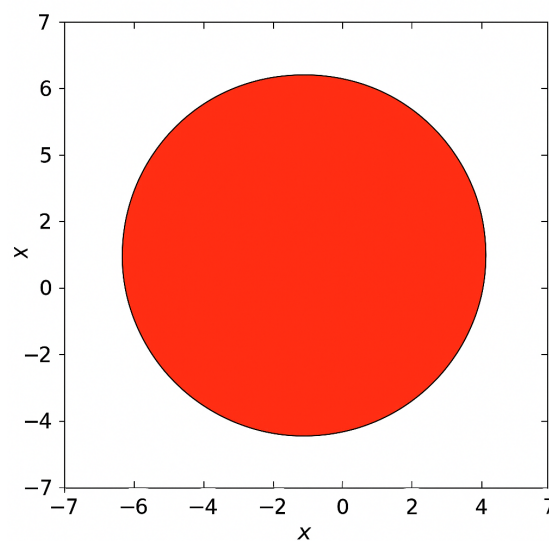


図 2: 半径 7 の円の面積

乱数によって生成されたプロットが円の内部に入る理論確率の計算は, 式 3 に示すように, 円の面積を全体の面積で割るだけである。本シミュレーションを簡略化するため, プロットの座標の上限および下限を -7 から 7 に制限し, 面積を 14×14 に設定する。

$$P = \frac{\text{円の面積}}{\text{領域の面積}} = \frac{\pi r^2}{A} \quad (3)$$

この場合、全体の領域の一辺が 14 であることが分かっているため、面積は $A = 14 \times 14 = 196$ と計算できる。一方、円の半径を 7 に設定しているため、円の面積は $\pi \times r \times r = \pi \times 7 \times 7 = 49\pi$ と計算できる。これら二つの値を式 3 に代入することで、式 4 を得り、乱数によって生成されたプロットが円の内部に入る確率は約 0.785 となる。

$$P = \frac{49\pi}{196} = \frac{\pi}{4} \approx 0.785 \quad (4)$$

2.1.3 課題 3: 異身長男女を一行に並べる

本実験では、身長異なる 10 名（男性 5 名、女性 5 名）をランダムに一行に並べ、各女子の左側に、自身よりも背の高い男子が 1 人以上いる並びとなる確率をシミュレーションにより求める。

各人の身長の関係は以下のように定義されており、すべての男性および女性は明確な順序で区別されている：

$$M_1 > W_1 > M_2 > W_2 > M_3 > W_3 > M_4 > W_4 > M_5 > W_5 \quad (5)$$

この関係に従って、10 名を一行にランダムに並べる。その上で、すべての女性について、「自身より背の高い男性が左側に少なくとも 1 人存在する」かどうかを調べる。この条件をすべての女性が満たす並びが、全体の中でどの程度の確率で現れるかを、シミュレーションにより統計的に推定する。

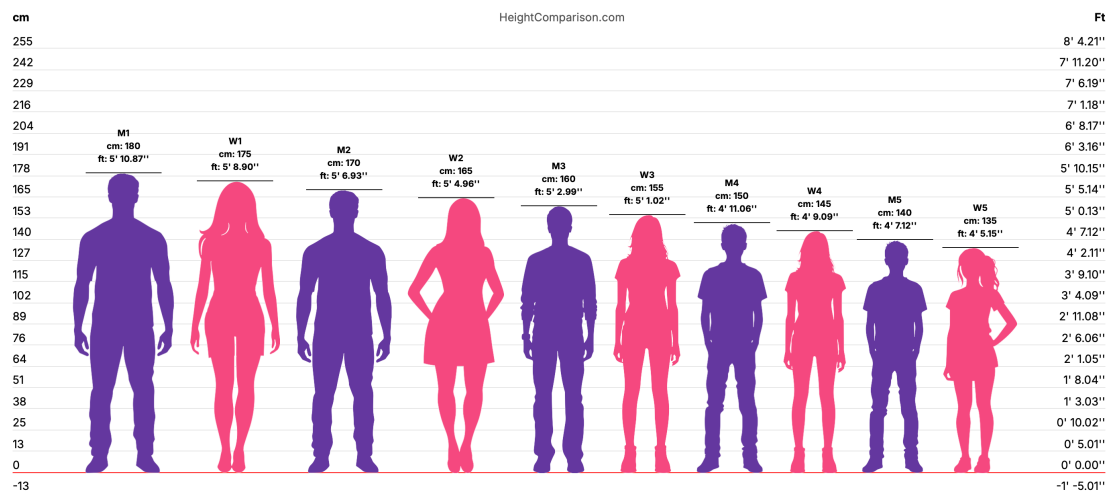


図 3: 男女 10 名の身長関係と識別

しかしながら、本問題の条件により、理論的な割合や期待値の計算は非常に複雑になる。先に述べた通り、列内の各女性に対して、少なくとも一人はその女性より身長の高い男性が左側に存在しなければならない。そのため、本シミュレーションでは 10 人の並びの全ての順列を総当たりで検証し、条件を満たすかどうかを確認する手法を用いる。

n 個の対象を一行に並べる場合の総順列数は、式 6 により計算できる。

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1 \quad (6)$$

今回のように、身長異なる 10 人（あるいは 10 個の対象）を一行に並べる場合、式 6 を用いることで、順列の総数は

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 3,628,800$$

通りであることが分かる。

次に、Python のコードを用いて、先に定義した条件に対して行のすべての順列を検証した。各順列をチェックするために使用した Python コードはコード 1 に示す。

コード 1: 順列を確認するコード

```
def is_valid(permutation):
    for i, person in enumerate(permutation):
        if person.startswith('W'):
            w_height = height_rank[person]
            left = permutation[:i]
            found_taller_man = any(p.startswith('M')
                                   and
                                   height_rank[p] < w_height for p in left)
            if not found_taller_man:
                return False
    return True
```

コードを各順列を確認する結果は表 1 に表されている。ブルートフォース法を用いて、条件を満たす順列の数が 893025 であることを確認した。有効な順列の数を全順列の数で割ることにより、理論的確率または期待値が 0.2461 となることが得られた。

表 1: 身長条件シミュレーションにおける順列結果

項目	値
総順列数	3,628,800
有効な順列数	893,025
確率（割合）	0.2461

2.1.4 課題 4: ジャンケンの割合

ジャンケンシミュレーションは、伝統的なジャンケンに得点制度を導入したシミュレーションである。提示された条件に従い、特定の手による勝敗によって得られる点数（本稿ではドル単位）が異なる。完全なルールは表 2 に示す。

表 2: じゃんけんの勝利手ごとの報酬（単位：ドル）

あなたの手 \ 相手の手	グー	パー	チョキ
グー (G)	0	-10	+10
パー (P)	+30	0	-30
チョキ (T)	-20	+20	0

ジャンケンに設定されたルールが複雑であるため、最も高い得点を得る各手（グー、チョキ、パー）の理論的な確率や期待される比率を正確に求めることはほぼ不可能である。しかし、式 7 を用いることで、その比率を近似的に求めることができる。

$$E = \sum_{i \in \{R, P, S\}} \sum_{j \in \{R, P, S\}} P_{\text{you}}(i) \cdot P_{\text{opp}}(j) \cdot \text{reward}(i \text{ vs } j) \quad \text{ここで } E \text{ は期待利益となる} \quad (7)$$

本初回のシミュレーションでは、相手の手は偏りなくランダムに選ばれる。そのため、各手の比率は正確に $\frac{1}{3}$ となる。先に表 2 で示した各手の勝敗に伴う得失を踏まえ、式 7 に代入することで、相手が偏りない場合に最も最適な勝率または戦略の比率を算出できる。

$$\begin{aligned}
E(r, p, s) &= r \cdot \left(\frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-30) + \frac{1}{3} \cdot (+10) \right) \\
&\quad + p \cdot \left(\frac{1}{3} \cdot (+30) + \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-20) \right) \\
&\quad + s \cdot \left(\frac{1}{3} \cdot (-10) + \frac{1}{3} \cdot (+20) + \frac{1}{3} \cdot 0 \right) \\
&= r \cdot \left(\frac{-30 + 10}{3} \right) + p \cdot \left(\frac{30 - 20}{3} \right) + s \cdot \left(\frac{-10 + 20}{3} \right) \\
&= r \cdot \left(\frac{-20}{3} \right) + p \cdot \left(\frac{10}{3} \right) + s \cdot \left(\frac{10}{3} \right) \\
&= \frac{1}{3} (-20r + 10p + 10s)
\end{aligned} \tag{8}$$

最大の利益 (E) を得るためには, $r + p + s = 1$ かつ $r, p, s \geq 0$ を満たす条件下で, $E(r, p, s)$ を最大化する必要がある. $r + p + s = 1$ であるため, 式 8 における s を $1 - r - p$ に置き換えることができ, これにより計算を式 9 の形に簡略化できる.

$$\begin{aligned}
E(r, p) &= \frac{1}{3} (-20r + 10p + 10(1 - r - p)) \\
&= \frac{1}{3} (-20r + 10p + 10 - 10r - 10p) \\
&= \frac{1}{3} (-30r + 10)
\end{aligned} \tag{9}$$

式 9 の内容を検討すると, r が増加するにつれて $E(r, p)$ の値は減少する. また, $E(r, p)$ を最大化する必要がある, $r \geq 0$ であることから, $E(r, p)$ は $r = 0$ のときに最大値をとる. 最後に, r を 0 に置き換えると, p と s の合計比率が 1 である条件のもとで, 式 10 に示すように期待利益は約 3.33 となる. この場合, p と s の任意の組み合わせ (例えば 0.2 と 0.8, 0.6 と 0.4 など) でも最大利益を得られることになる.

$$E(0, p) = \frac{1}{3} (0 + 10) = 3.33 \tag{10}$$

式 8 から式 10 にかけて行ったすべての計算は, 相手が特定の手を他の手よりも頻繁に出すといった偏りを持たないという前提に基づいている. この前提のもとでは, 各手 (グー, チョキ, パー) はそれぞれ約 0.33 の確率で出現すると仮定される.

しかし現実においては, 人間は学習を通じて自身の出す手の比率を調整し, 最大限の利得を得ようとする傾向がある. このような状況下では, 最適な手の出し方の比率を求めることがより複雑になる. というのも, 相手の勝利戦略自体が繰り返しの中で動的に変化するためである.

本実験では、より現実に近い状況を模擬するために、相手に機械学習を導入し、完全なランダムではない戦略を用いる。こうした環境下では、理論的に最適な勝利戦略を計算することがほぼ不可能であるため、モンテカルロ・シミュレーションの重要性がより一層高まる。

2.1.5 課題 5: ビンゴのシミュレーション

本シミュレーションでは、表 3 に示した条件に従い、ランダムに生成されたビンゴカードを作成する。

表 3: ビンゴカードの列と対応する数値範囲

列	数値範囲
1 列目 (B)	1 ~ 15
2 列目 (I)	16 ~ 30
3 列目 (N)	31 ~ 45
4 列目 (G)	46 ~ 60
5 列目 (O)	61 ~ 75

また、従来のビンゴのルールでは、フルビンゴを達成するために 5 列すべてを完成させる必要がある（各列につき 1 つ）。しかし本シミュレーションでは、1 列の完成をフルビンゴとみなす。完成した列は、ビンゴカード上の任意の位置における横、縦、または斜めのいずれかとなる。また、ビンゴカードの中央はフリースペースとされ、ゲーム開始時点で既にこのマスは開いているものとする。

ビンゴゲームにおいて、以下の 3 つの異なる条件がシミュレーションされた。

1. 7 回以内で当たる確率をシミュレーション

本シミュレーションは、7 回のコール以内当たる確率を求めることを目的とする。

2. 最短回数 (4 回) で当たりとなる確率をシミュレーション

本シミュレーションは、最短ターン数である 4 回以内（ビンゴカード中央が既に開いていることを考慮）に当たる確率を算出することを目的とする。

3. 当たりとなる最も遅いゲーム回数とそのような状態の発生確率をシミュレーション

本シミュレーションは、最も遅い（コール数が最大となる）タイミングで当たる確率を算出することを目的とする。ビンゴカードには、表 3 に示した通

り, 75 の候補番号のうち 24 個 (5×5 のマスから中央の 1 マスを除く) が記載されている. これは, ビンゴカード上の番号に当たらずに呼ばれることが可能な番号が $75 - 24 = 51$ 個存在することを意味する. ビンゴカードにない番号をすべて呼んだ後でも, 表 4 に示すように, ビンゴカード上で列が完成しないまま呼べる番号がさらに 19 個存在する.

表 4: 完全列または表を当たらないビンゴカード

B	I	N	G	O
O	X	X	X	X
X	X	O	X	X
X	X	X	O	X
X	O	X	X	X
X	X	X	X	O

つまり, 列や行が完成しないまま呼べる番号の総数は $51 + 19 = 70$ となる. 言い換えれば, 最悪の場合でも 71 回目のコールで必ずビンゴが達成されることになる.

これらのシミュレーションにおける正確な理論的確率や期待値を計算することは極めて複雑であり, 単一の数値を導出するための単一の方程式は存在しない.

しかし, 表 3 に示したビンゴカード作成のルールを考慮すると, 式 11 に示す組み合わせの公式を用いて, 可能な組み合わせ数を計算することができる.

$$nCr(n, r) = \frac{n!}{r!(n-r)!} \quad (11)$$

$$\text{B 列: } 15C5 = 3003$$

$$\text{I 列: } 15C5 = 3003$$

$$\text{N 列: } 15C4 = 1365 \quad (\text{中心の FREE スペースのため 4 つ選ぶ})$$

$$\text{G 列: } 15C5 = 3003$$

$$\text{O 列: } 15C5 = 3003$$

(12)

$$\text{合計のカード数: } (15C5)^4 \times 15C4 = 3003^4 \times 1365$$

$$\begin{aligned} \text{数値計算: } 3003^4 &\approx (3.003 \times 10^3)^4 = 3.003^4 \times 10^{12} \approx 81.2 \times 10^{12} \\ &= 8.12 \times 10^{13} \end{aligned}$$

$$8.12 \times 10^{13} \times 1365 \approx 1.1 \times 10^{17}$$

したがって、式 12 の計算により、ビンゴカードの総数は約 1.1×10^{17} 。この総数で条件を満たすカードを計算することは本手法は多大な計算資源を要する。それにより、各シミュレーション条件の確率を近似するためには、シミュレーションを用いる方が望ましい。

2.2 シミュレーションの役割

現実には、与えられた条件に対する理論的確率や期待値の導出は、ルールの変更やバイアスの導入によって困難になる場合がある。例えば、節 2.1.1 および 2.2.2 で示したように、6 面ダイスや単純な 2 次元円を用いた場合の理論的確率の算出は、シミュレーション自体で考慮すべき要素が少ないため容易である。しかし、ダイスに偏りがあり、特定の面が他の面よりも出やすいといった条件を導入した場合、理論的にはその偏りの度合いが正確に分かっていれば、バイアスを計算し理論値を求めることは可能である。とはいえ、実際には複数の予期しない要因が結果に影響を及ぼす可能性があり、このような解析ははるかに困難となる。[6]

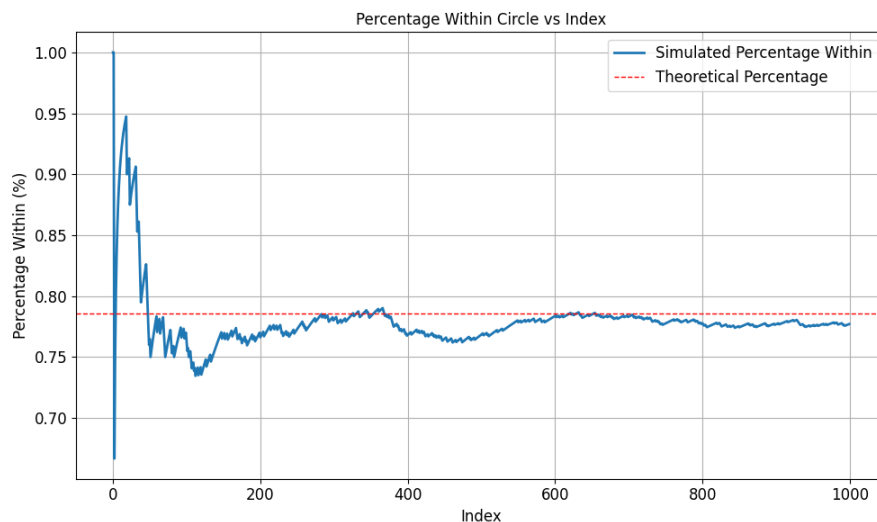


図 4: 出現率が収束する例

このような場合において、理論的確率を直接計算するのではなく、シミュレーションを用いることがはるかに有効な代替手段となる。シミュレーションでは、条件に従って同じ実験を何度も繰り返し実行し、有効な結果のみを記録すればよい。シミュレーションの回数を十分に重ねることで出現率は次第に安定し、図 4 に示すように、当該条件における出現確率のおおよその傾向を把握することが可能となる。

2.3 モンテカルロシミュレーション

モンテカルロシミュレーションとは、複雑な条件における確率を得るために繰り返しサンプリングを用いる計算アルゴリズムの一種である。節 2.2 で述べたように、条件によっては理論的な確率を正確に計算することが計算コストの面で非常に高く、多大な時間を要する場合がある。モンテカルロシミュレーションを用いることで、すべての順列を網羅的に列挙することなく、不確実な事象の発生確率を推定することが可能となる。たとえば、節 2.1.5 で定義された制約下におけるビンゴのシミュレーションでは、ビンゴカードの可能な順列数が 1.1×10^7 以上存在するため、全順列を評価するには数日から場合によっては数年を要する可能性がある。このような場合にモンテカルロシミュレーションを用いることで、当該事象の確率を現実的な計算時間内で推定することができる。[2]

3 実験方法

本節では、節 2.1 で述べた実験において使用される各種手法および関数について説明する。

3.1 ハードウェアと環境

ハードウェアとしては Apple シリコンのプロセッサに基づいて実験を行う。

機種名 Apple Macbook Pro (M3, 2023 年モデル)

プロセッサ Apple M3 チップ (8 コア：高性能コア 4 + 高効率コア 4)

GPU 10 コア統合型 GPU

メモリ 16GB ユニファイドメモリ

ストレージ 1TB SSD

OS macOS Sequoia 15.5

アーキテクチャ ARM64 (Apple シリコン)

また、作成した実験プログラムは C 言語で書き、GCC コンパイラでコンパイルされる。

Apple clang version 17.0.0 (clang-1700.0.13.5)

Target: arm64-apple-darwin24.5.0

Thread model: posix

コンパイルを効率化するため、GCC コンパイラを Make と同時に利用する。

GNU Make 3.81

Copyright (C) 2006 Free Software Foundation, Inc.

This is free software; see the source for copying conditions.

There is NO warranty; not even for MERCHANTABILITY or
FITNESS FOR A PARTICULAR PURPOSE.

This program built for i386-apple-darwin11.3.0

3.2 ランダム関数

C 言語において、乱数値を生成する最も一般的な方法の一つは、`stdlib.h` ライブラリが提供する `rand()` 関数である。`rand()` 関数は通常、乱数の種（シード）を設定するための `srand()` 関数と組み合わせて用いられる。一般に「乱数」とは、次にどの値が出るか予測できないことを意味するが、広く使われている `rand()` 関数の実装は厳密には真の乱数ではなく、疑似乱数である。先に述べたように、`rand()` 関数は `srand()` 関数と組み合わせて使用され、乱数生成の開始点となる種を設定する。この種は、疑似乱数生成の計算基準となる開始値である。この開始値は式 13 におく LCG (Linear Congruential Generator) であり、予め定められた定 x 数群を用いて次の疑似乱数を生成するために用いられる。

$$X_{n+1} = (aX_n + c) \bmod m \quad (13)$$

この式は `rand()` 関数内に実装されており、コード 2 に類似した形で表される。

コード 2: `rand()` 関数の简单版

```
static unsigned long next = 1;

int rand(void) {
    next = next * 1103515245 + 12345;
    return (unsigned int)(next / 65536) % 32768;
}

void srand(unsigned int seed) {
    next = seed;
}
```

一定の式を用いているため、乱数生成は開始種が分かっているならば、実際には生成される値を予測可能である。これは、`stdlib.h` ライブラリが提供する一般的な `rand()`

関数はエントロピーが低く、 m 回の出力後には必ず生成される値が繰り返されることを意味する.[1]

モンテカルロシミュレーションにおいて、この制約はシミュレーションの精度を著しく低下させる。なぜなら、可能なすべてのケースや順列を十分に一般化するために、シミュレーションを可能な限り多く繰り返す必要があるからである.[8]

この制約を回避するために、すべてのシミュレーションは、`stdlib.h` ライブラリが提供する `arc4random` の拡張である `arc4random_uniform` 関数を用いて実行する。`rand` 関数とは異なり、`arc4random` は LCG（線形合同法）を使用せず、より安全な CSPRNG（暗号学的に安全な疑似乱数生成器）を採用している。`rand` と `arc4random` はいずれも真の乱数ではなく疑似乱数であるが、`arc4random` はデバイスのハードウェアから得られるエントロピー値を乱数の種として利用する点異なる。一方で、一般的な `rand` 関数はエポック時刻を基に初期シードを設定しているため、実行時刻が正確にわかれば、次に生成される乱数を「予測」できる可能性がある.[9]

`arc4random` 関数を実行すると、デバイスは `/dev/urandom` から疑似乱数バイト列を読み取る。この `urandom` ファイルは、デバイスのカーネルが管理する内部エントロピープールから生成される。エントロピープールは、ネットワークタイミング、キーボードやマウスの入出力、割り込みタイミングなど、ハードウェアに基づくさまざまな値からシードされる。`/dev/urandom` から読み取った内容の一例をコード 3 に示す。

コード 3: `urandom` の内容のを読み取り結果例

```
head -c 16 /dev/urandom | xxd
00000000:
e690 1de3 6b5c 6c27 7684 fc82 6cfa a9a3
....k\l'v...l...
head -c 16 /dev/urandom | xxd
00000000:
c15c a4e2 c72e 8d0d 5494 4f21 874b d0b3
.\.....T.O!.K..
head -c 16 /dev/urandom | xxd
00000000:
3198 d4b2 7c21 5e2b 41a6 7a94 8012 f0ab
1...|!^+A.z.....
```

これらのランダムな値は、ChaCha20 アルゴリズムに入力される。ChaCha20 は、

Daniel J. Bernstein によって 2008 年に設計されたストリーム暗号であり、実用的な暗号解析に対して高い耐性を持つ。現代の暗号技術において広く用いられており、OpenSSH や TLS (Transport Layer Security) , 各種オペレーティングシステムなど、業界標準の技術においても利用されている.[7]

arc4random は、低レイヤのハードウェア層から生成されるエントロピー値を元にシードが設定されるという特性を持つ。このため、arc4random は通常の rand 関数と比較して、より「ランダム」に近い値を生成することが可能となる。

3.3 フィッシャー–イェーツのシャッフル

Fisher-Yates シャッフルは、配列内の要素を重複なしでランダムに並び替える代表的なアルゴリズムである。配列の長さが 10 である場合、各要素をランダムに生成して代入する方法では、重複が発生する可能性があるため適切ではない。この問題を回避するため、まず配列に 0 から 9 までのあらかじめ定義された値を格納し、その後ランダムなインデックスを用いて 2 つの要素を入れ替える処理を繰り返すことで、全体をランダムにシャッフルする。

このアルゴリズムでは、配列の末尾から順に走査しながら、現在の要素とそれ以前の任意の要素をランダムに選んで交換する。これにより、全ての順列が等確率で生成されるという性質を持つ。Fisher-Yates シャッフルを実装したコードについては、code4 に示す.[3]

この方法は、課題 3 で異なる身長 of 10 人の列を繰り返しなくシャッフルする際に使用する。

コード 4: フィッシャー–イェーツのシャッフルコード

```
for (int i = n - 1; i > 0; i--) {  
    int j = arc4random() % (i + 1);  
    swap(arr[i], arr[j]);  
}
```

3.4 モジュロによるランダム関数のバイアス

ランダム値ジェネレータ内部には、値をユーザに返す前に行われる最終的な操作が存在する。この操作はモジュロチェックと呼ばれ、生成された値に対してあらかじめ設定された最大値で剰余演算を行うことで、値がその範囲を超えないように制限される。しかし、この手法はランダム値を特定の範囲内で生成したい場合に大き

な問題を引き起こす.[4] 例えば, ランダム値ジェネレータが符号なし 8 ビット整数の最大値である 255 まで出力可能だとする. このとき, `arc4random` 関数に対して最大値を 120 と指定した場合, 120 を超えるすべての数値は先頭から再び数えられるように変換される. これにより, 予期せぬバイアスが発生し, 特定の値が他の値よりも頻繁に出力される傾向を持つようになる.

`arc4random` と `arc4random_uniform` の違いを適切に評価するために, C 言語を用いて小規模なシミュレーションを実施した. `arc4random` は上下限の設定を直接サポートしないため, コード 5 に示すように再実装を行った.

コード 5: 再実装した `arc4random` 関数

```
int rand_in_range(int min, int max){
    return arc4random() % (max - min + 1) + min;
}
```

一方, `arc4random_uniform` は下限をサポートしないため, コード 6 に示すようにこちらも再実装を行った.

コード 6: 再実装した `arc4random_uniform` 関数

```
int rand_uniform_in_range(int min, int max){
    return arc4random_uniform(max - min + 1) + min;
}
```

両関数を下限 1, 上限 100 で 10 万回繰り返し実行した結果は, 図 5 に示すように類似している.

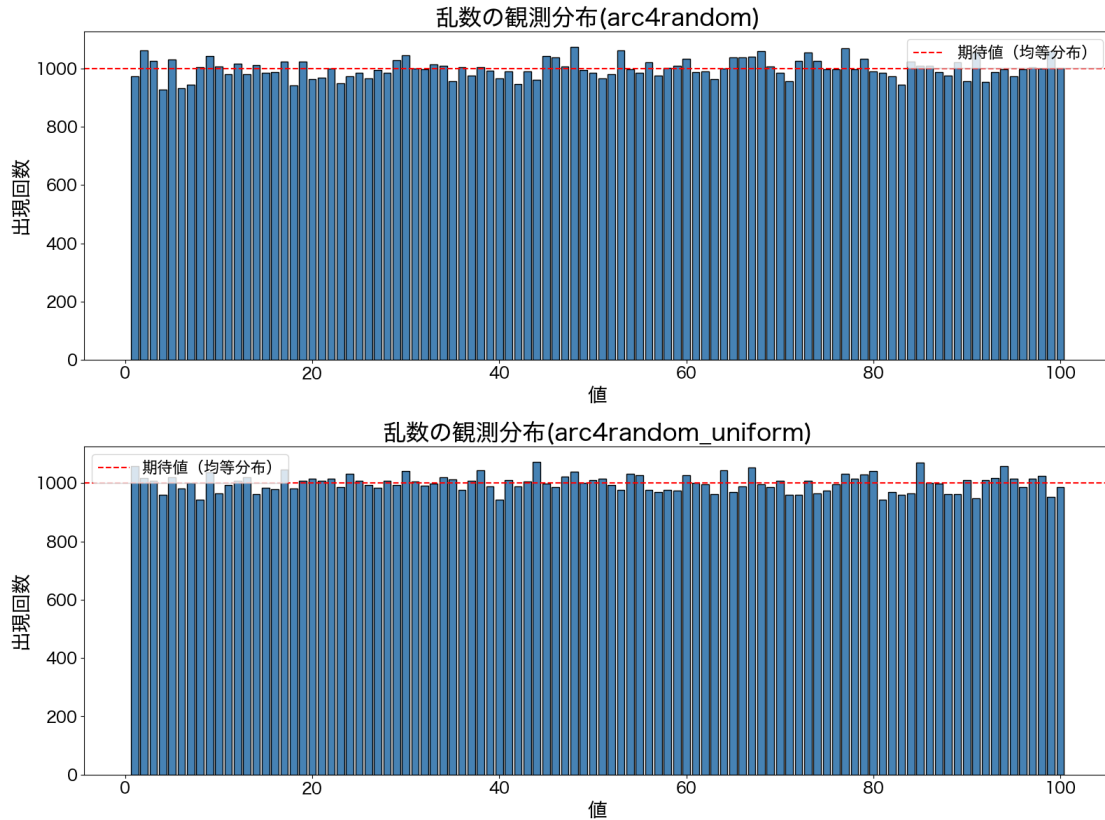


図 5: arc4random と arc4random_general によって生成された乱数の比較 (下限 1, 上限 100)

グラフをそのまま比較した場合, 2つの関数間に有意な差があるとは言い難い. 適切な比較を行うために, 両手法の p 値およびカイ二乗値を算出する.

表 5 の結果から, モンテカルロシミュレーションに必要な高いエントロピーを arc4random_uniform の結果において確認できる. エントロピーが高いほど, 生成される値はよりランダムであることを示す. 一方で, カイ二乗値が低いことは, この結果が一様分布に近いことを意味する.

両関数とも p 値に基づくバイアスは有意ではないものの, arc4random_uniform のエントロピースコアが高いことにより, シミュレーションデータに含まれる潜在的なノイズが除去される. 以上の理由から, 本実験ではすべてのシミュレーションに arc4random_uniform 関数を用いる.

表 5: カイ二乗検定の結果

メソッド	カイ二乗統計量	P 値
<i>arcRandom</i>	103.030000000	0.370758315
<i>arcRandom_uniform</i>	83.980000000	0.859645089

3.5 実行数

前節 2.3 で述べたように, モンテカルロシミュレーションでは理論値に近い結果を得るために大量の反復回数が必要となる. サイコロのような単純なシミュレーションでは 1000 回の反復で十分な場合もあるが, より複雑なルールセットや稀な事象を扱うシミュレーションでは, 正確な結果を得るために 100 万回以上の反復が必要となることがある. 適切な反復回数の設定は困難であり, 多くの試行錯誤を要する. 繰り返し回数の変化による影響は, 図 6 と図 7 に示されている. この図では, 2つのサイコロが毎回同じ目を出す確率のシミュレーションをそれぞれ 100 回および 1000 回繰り返した結果を比較している.

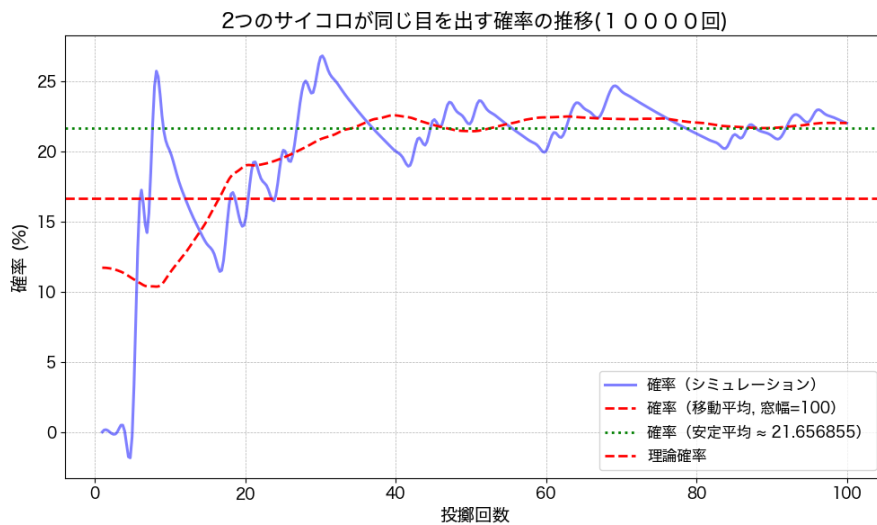


図 6: 二つのサイコロが同じ目を出す確率の比較 (100 回)

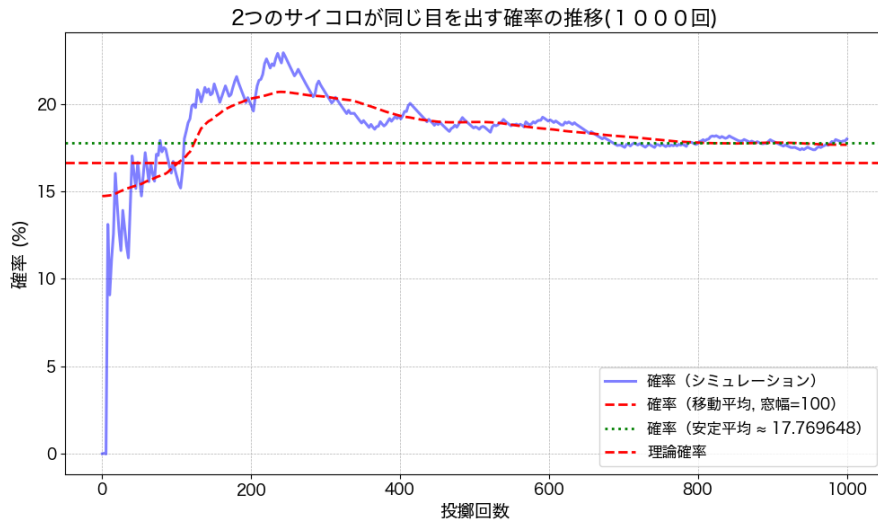


図 7: 二つのサイコロが同じ目を出す確率の比較 (1000 回)

2つのサイコロが1回の試行で同じ目を出す理論的な確率は、単純に以下のように計算できる:

$$\frac{6}{6} \times \frac{1}{6} = \frac{1}{6} \approx 0.167 \approx 16.7\%$$

つまり、モンテカルロ・シミュレーションによっても同様の値が得られるはずである。異なる繰り返し回数で実施した同一シミュレーションの比較結果は、表6に示されている。

表 6: 投擲回数, 安定平均, および理論値 16.7%からの偏差率の比較

投擲回数	安定平均 (%)	理論値 16.7%からの偏差率 (%)
100	21.656855	$\frac{ 21.656855 - 16.7 }{16.7} \times 100 \approx 29.72$
1000	17.76948	$\frac{ 17.76948 - 16.7 }{16.7} \times 100 \approx 6.44$
10000	16.542044	$\frac{ 16.542044 - 16.7 }{16.7} \times 100 \approx 0.95$

この簡単な実験結果から、繰り返し回数が多くなるほど理論的な期待値に近づくことが確認できる。しかしながら、任意のシミュレーションに対して最適な繰り返し回数を一意に決定する標準的な手法は存在しない。

したがって、本シミュレーションで扱うより複雑な実験においては、繰り返し回数を手動で調整し、移動平均による値の収束が最も安定するように設定を行う。

3.6 移動平均による値の収束

移動平均は、統計解析および時系列処理における基本的な手法である。この手法では、あらかじめ定めたウィンドウサイズに基づいて、時系列データ上をスライドさせながら局所的なデータ区間の平均値を計算する。

この処理により、短期的な変動が平滑化され、より長期的な傾向や周期性が浮き彫りになる。移動平均は、ノイズの多いデータから本質的なパターンを抽出する際に有効である。移動平均を用いることで、データに含まれる短期的なノイズを平滑化し、全体的な傾向とは無関係な変動を除去することができる。これにより、データ内の有意なパターンや潜在的なトレンドが明確に現れるようになる。[5]

移動平均の計算には、最も基本的な形態である単純移動平均 (Simple Moving Average, SMA) を用いる。SMA の計算式は式 14 に示す通りである。この式は、直近の w 個のデータを入力として受け取り、それらの平均値を一つの値として算出する。データを順次処理しながら SMA を繰り返し計算することで、シミュレーションの比率変動を平滑化する。

$$\text{SMA}_t = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i} \quad (14)$$

3.7 ジャンケンシミュレーションに用いる機械学習

前述のとおり (節 2.1.4 参照)、じゃんけんにおける現実に近い状況をシミュレートするため、機械学習を導入し、繰り返しごとに戦略を最適化する学習型の対戦相手を構築する。

本実験で使用するモデルは、以下の特性を持つように設計されている:

特徴ベクトル じゃんけんにおける次の手を予測するための 6 次元の入力ベクトル。以下の 2 種類の特徴量から構成される。

直前の手 (3 次元): 相手 (自分側) が最後に出した手を one-hot 表現したもの。グーであれば $[1, 0, 0]$ 、パーであれば $[0, 1, 0]$ 、チョキであれば $[0, 0, 1]$ と表現される。

セッション内の手の比率 (3 次元): セッション全体で相手 (自分側) が出した手の分布。各値は 0 から 1 の間で、3 つの値の合計は 1 になる。

モデル出力 次に出す手 (グー・パー・チョキ) それぞれに対応するスコアまたは確率を示す 3 次元の出力ベクトル。

グースコア: 次にグーを出す確率または信頼度。

パースコア: 次にパーを出す確率または信頼度.

チョキスコア: 次にチョキを出す確率または信頼度.

モデル構造 ソフトマックス関数を適用した単一の線形層で構成される. 隠れ層は存在せず, モデル構造は単純な多クラスロジスティック回帰に相当する. 可読性と学習の安定性を重視して設計されている.

4 実験結果

4.1 課題 1: 7つのサイコロが同じ目を出すシミュレーション

前節 2.1.1 で説明した通り, 7つのサイコロを投げた際に全て同じ目が出る理論的確率は約 2.143×10^{-5} である. 異なる投擲回数における出現率のサンプリング結果は表 7 に示されている.

表 7: 指定投擲回数における生データの出現率と理論確率に対する偏差率 (%)

投擲回数	出現率 (%)	偏差率 (%)
50,000	0.000000	-100.00
100,000	0.400000	86.74
500,000	0.240000	11.99
1,000,000	0.300000	39.96
5,000,000	0.222000	3.58
理論確率	0.214335	—

対象条件の出現率は, その時点での出現回数をそれまでの投擲回数で割ることにより計算される. 5つの異なる時点 (50,000 回から 5,000,000 回) でのサンプル率を取得した結果, 7つのサイコロが同じ目を示す出現率は, 前節で計算した理論値または期待値 2.143×10^{-5} に徐々に収束することが確認された.

繰り返し回数が増加するにつれて, 出現率はより理論値に近づいていく様子が, 図 8 に示されている.

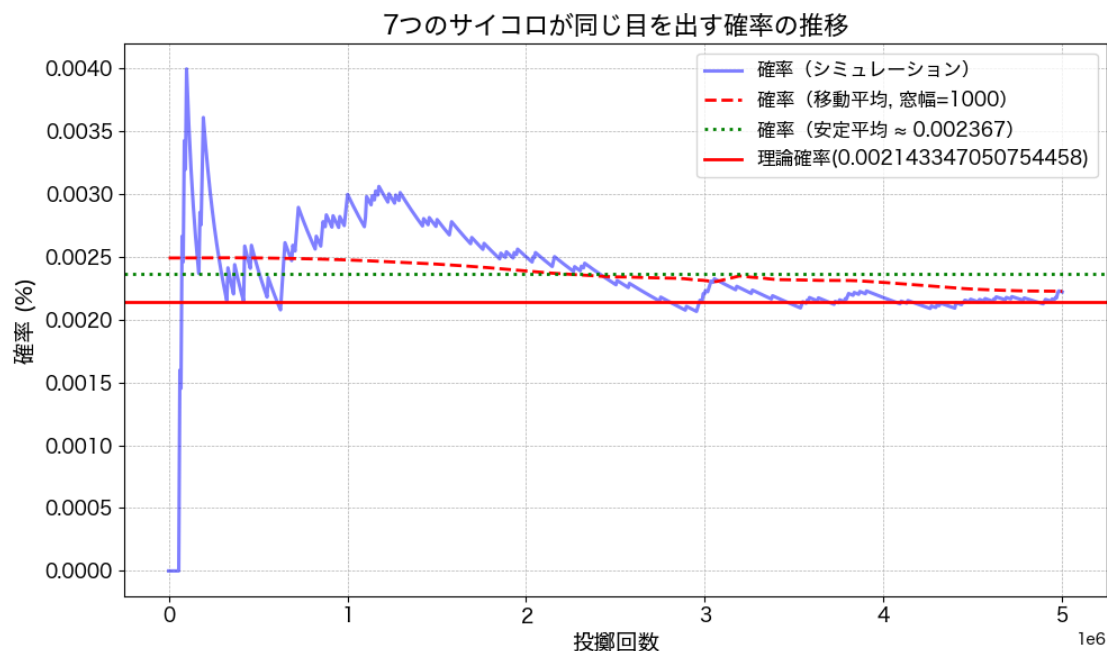


図 8: 投擲回数に対する 7つのサイコロが同じ目を出す確率

図 8 に示すように、投擲回数が少ない段階では出現率に大きな変動と揺らぎが見られる。しかし、繰り返し回数が増加するにつれて、グラフは安定し、変化率も著しく低下する。

この減速傾向にもかかわらず、窓幅 1000 データポイントの単純移動平均により強調された全体的な傾向は、理論的な期待確率へと着実に収束している。

1 回の投擲におけるサイコロの個数を変更した際の挙動は、表 8 に示されている。

表 8: サイコロの個数ごとの一致確率とその偏差 (5 百万回)

サイコロの個数	理論確率 (%)	実測値 (%)	偏差 (%)
2	16.666666667	16.667500000	+0.00500
4	0.462962963	0.462260000	-0.15184
6	0.012860082	0.012360000	-3.88864
8	0.000357225	0.000580000	+62.36288
10	0.000009923	0.000500000	+4938.84800

表 8 からわかるように、サイコロの個数が 2 個、4 個、6 個である場合、500 万回のシミュレーションによって得られた出現率は、計算された理論確率または期待値

からの偏差が5%未満に収まっている。特に、2個および4個のサイコロを用いたシミュレーションでは、最終的な偏差は0.5%未満にまで抑えられている。

一方で、サイコロの個数が8個および10個となるシミュレーションでは、理論値からの偏差が著しく大きく、最終的な偏差はそれぞれ約62.36%、および4938.85%に達している。

4.2 課題2: 半径7の円の面積

節4.1の実験と同様に、本シミュレーションにおいても、ランダムな点を繰り返し生成し、そのうち何点が円の内部に含まれるかをカウントすることで確率を算出している。各繰り返し時点において、円内に収まった点の総数をその時点までの繰り返し回数で割ることで、出現比率が計算される。

異なる繰り返し回数における、円内に収まる点の比率または割合のサンプリング結果は、表9に示されている。

表9: 繰り返し回数ごとの内包率と理論値との偏差

試行回数	実測内包率 (%)	理論内包率 (%)	偏差 (%)
500 回	80.800000	78.539816	+2.8778
700 回	80.142857	78.539816	+2.0411
1000 回	80.600000	78.539816	+2.6231
2000 回	78.650000	78.539816	+0.1403
3000 回	79.500000	78.539816	+1.2225
4000 回	78.850000	78.539816	+0.3949

表9のシミュレーション結果を見ると、比較的少ない繰り返し回数である500回においても、偏差率は約2.88%に抑えられていることがわかる。さらに繰り返し回数を増加させることで、偏差率は徐々に低下し、4000回時点ではわずか0.39%にまで縮小した。

このシミュレーション結果をグラフ化したものは、図9に示されている。

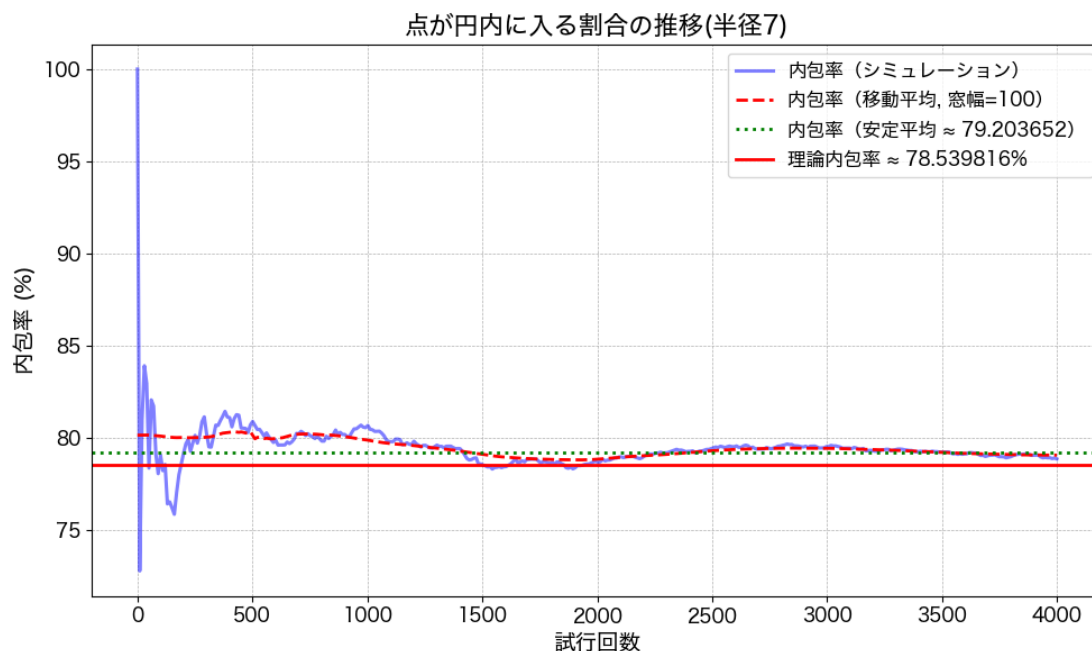


図 9: ランダム点が円内に属する割合 (半径 7 の円の面積)

図 9 に示すように、理論値からの偏差率は 5% 未満と比較的小さいものの、繰り返し回数が 1000 回程度の低い段階では、円内に点がプロットされる割合は依然として不安定である。しかし、繰り返し回数が 2000 回に近づくと、円内に収まる点の割合は大きく低下し、計算された理論値との偏差はわずか 1.22% にまで縮小した。

図 9 の 2000 回付近に着目すると、比率の推移はなお不安定であり、2500 回付近では再び理論値からの乖離が見られる。この段階以降、グラフの変動は徐々に安定しはじめるが、移動平均 (Simple Moving Average, SMA) を窓幅 100 で適用した線を通じて、理論値へと緩やかに収束し続けている様子が確認できる。

図 10 を通じて、2 次元空間内にプロットされた点の分布を観察できる。このグラフから、生成されたランダムな点はあらかじめ設定された範囲 -7 から 7 にかけて、均等かつ広く分布していることが確認できる。

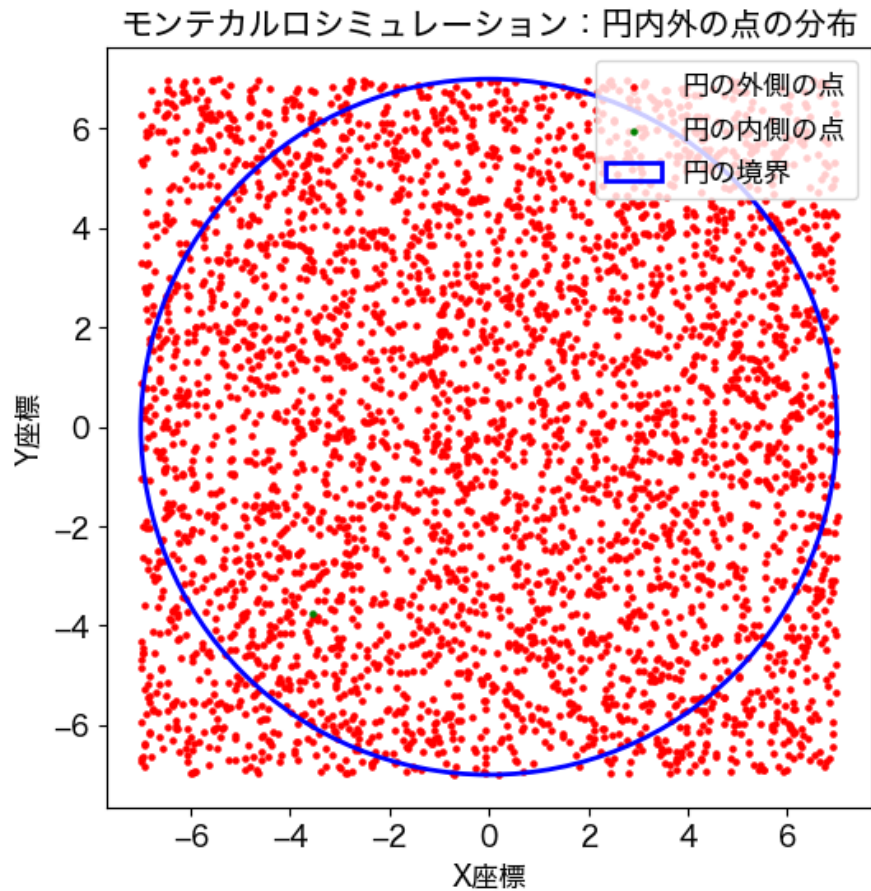


図 10: 2次元空間内にランダムにプロットされた点（半径7の円内）

シミュレーション範囲（面積）の変化による影響を、繰り返し回数を一定に保ったまま観察するために、円の半径を2, 4, 6, 8, 10, および20に設定して実験を行い、それぞれ4000回の繰り返しを行った。結果は表10に示されている。

表 10: 半径ごとの内包率と理論値との差

半径 (r)	内包率 (%)	理論値 (%)	偏差 (%)
2	78.400000	78.539816	-0.1398
4	80.200000	78.539816	+1.6602
6	78.500000	78.539816	-0.0398
8	78.100000	78.539816	-0.4398
10	79.450000	78.539816	+0.9102
20	78.025000	78.539816	-0.5148

表10に示す通り、円の半径を変えた各シミュレーションにおける理論的比率は78.54%で一定である。シミュレーションの半径を変化させた結果、期待値からの

偏差は1%未満に収まっている。しかし、半径4の円で実施したシミュレーションでは、期待値に対する偏差が突然1.66%を超える跳ね上がりを示した。これはデータのノイズによるものであり、結果として重要な意味を持つものではないと考えられる。

4.3 課題3: 異身長男女を一行に並べる

先に節2.1.3で計算した通り、身長の異なる5人の男性と5人の女性、合計10人を一行に並べる場合の全通り数は3628800通りである。そのうち、各女性の左側に少なくとも1人は自分よりも背の高い男性が存在するという条件を満たす並びはわずか893025通りしか存在しない。つまり、条件を満たす有効な並びの割合は全体のわずか24.61パーセントである。

このシミュレーションでは、同様の条件に従って1万回の試行を行う。各試行では、並びが条件を満たすかを判定し、有効な並びであればカウントを増やす。この有効な並びの累積数を、各時点での試行回数で割ることで、推定値（ランニングエスティメート）を算出する。1万回のシミュレーションデータのうち、複数の点における推定値をサンプリングした結果を表11に示す。

表 11: 指定試行回数における推定確率と理論確率に対する偏差率（%）

試行回数	推定確率（%）	偏差率（%）
50	19.4481	-20.99
100	20.9931	-14.70
500	22.1017	-10.19
1,000	23.1609	-5.89
2,000	24.2689	-1.39
5,000	24.9405	+1.34
10,000	25.0573	+1.82
理論確率	24.6100	—

表11から分かるように、試行回数が増加するにつれて、ランニングエスティメートの理論値に対する乖離率は徐々に減少していく。試行回数が2000回を超えると乖離率は5パーセント未満に収束し、おおよそ1.3%前後で推移するようになる。

このシミュレーション結果をグラフ化したものを図11に示す。

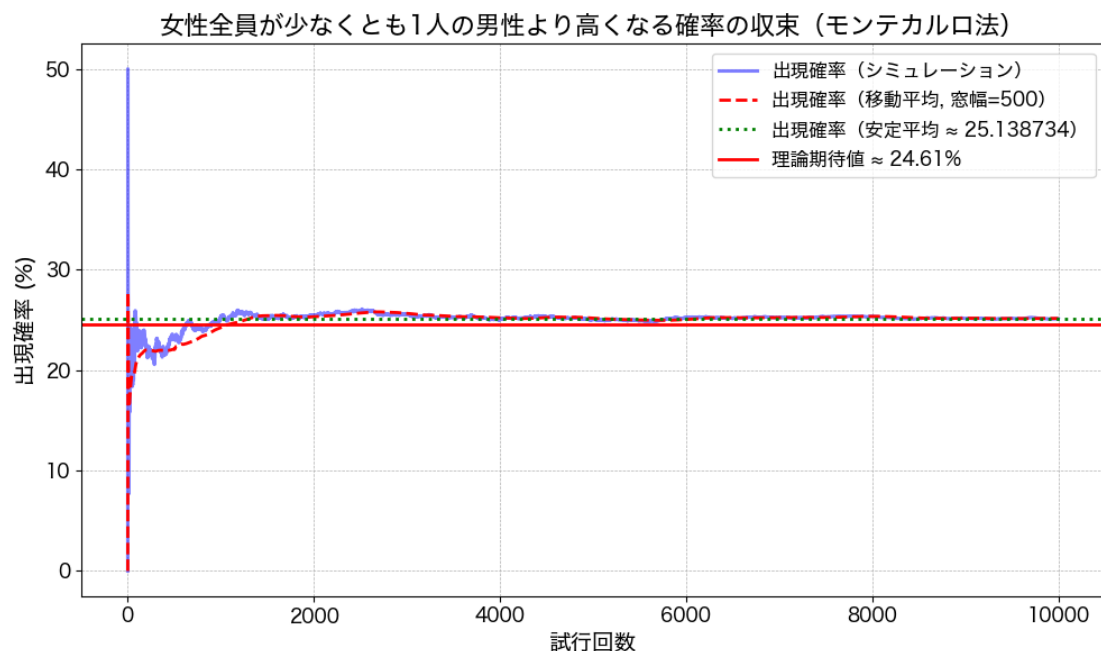


図 11: 女性の身長が少なくとも 1 人の男性を上回る確率の推定値と理論値との偏差（各試行回数時点）

図 11 に示すように、ランニングエスティメートの線は試行回数が 2000 回に到達する前から理論値へと収束し始めていることが確認できる。それ以前にも一時的な収束は見られるが、線は依然として不安定で大きな変動を伴っていた。一方で、2000 回以降は線が安定し、期待値に向けて緩やかな下降傾向を示すようになる。

最終的に、8000 回以降も完全な収束には至っていないものの、ランニングエスティメートは 1.3% から 1.8% の範囲で理論値に近い値を維持しており、これらの乖離は誤差範囲（5 パーセント未満）とみなせるため、データのノイズによる影響であると考えられる。

4.4 課題 4: ジャンケンの割合

前述のセクション 2.1.4 で述べたように、特定の手で勝利した場合に異なる報酬が得られる独自ルールじゃんけんにおいて、最も利益の高い手の比率を求めるためにはシミュレーションが必要となる。しかしながら、最適な手の比率は相手の手の出し方に大きく依存する。そのため、本節では 2 つの異なる実験を行う。1 つ目は相手が完全にランダムで手を出すと仮定した場合、2 つ目は簡易的な機械学習を用いて相手がこちらの戦略を学習して対抗する場合である。

4.4.1 相手が完全にランダムな場合（バイアスなし）

本シミュレーションでは、相手が以下のように均等な確率で手を出すと仮定する：

$$\text{グーの出現率} = \text{チョキの出現率} = \text{パーの出現率} = \frac{1}{3}$$

最適な手の比率を見つけるために、各手の比率を 0.01 刻みで全探索し、各比率に対してシミュレーションを 100 000 回繰り返して実行した。各回で勝利回数、獲得ポイント（ドル）、および平均獲得額を計算した。最終的な各比率に対するデータを集計し、他の比率と比較することで、最も高い利益が得られる手の出し比率を決定した。その結果を図 12 に示す。

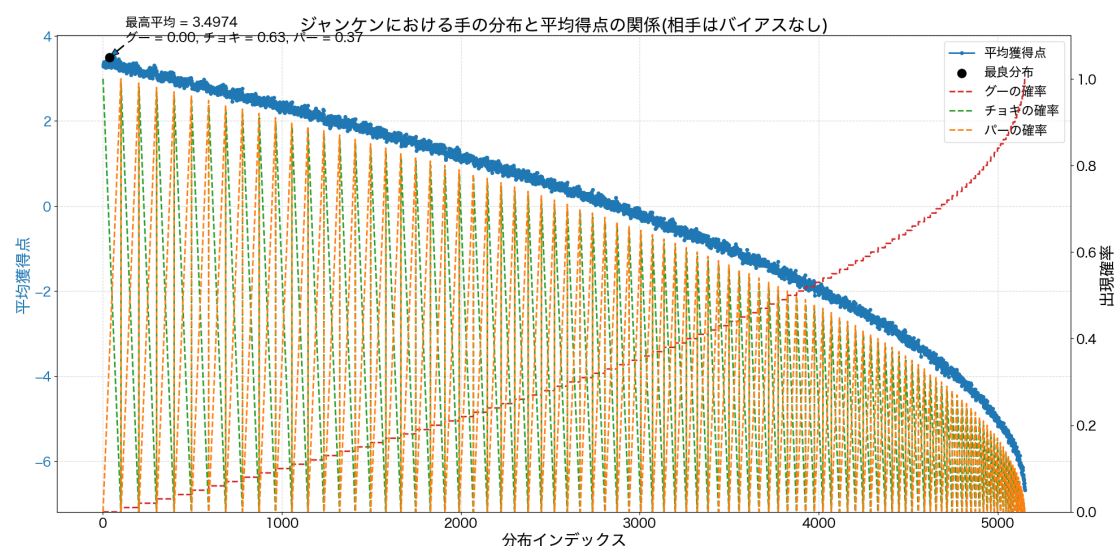


図 12: 手の出し比率による比較（バイアスのない相手）

図 12 より、相手が完全にランダムで手を出すと仮定した場合に最も良い結果を出す手の出し比率は、表 12 の通りである。

表 12: 相手が偏りのない戦略を取ると仮定した場合の最適な手の比率

手の種類	出現率 (%)
グー	0
パー	37
チョキ	63

シミュレーションプログラムから出力されたデータを表 13 に示す。表 12 で得られた最適比率を用いた場合、平均獲得額は 3.4974、総獲得額は 349 740 であること

が確認できる. 100 000 回のシミュレーションのうち, 合計 33 747 回の勝利が確認された. これは全体の 33.74%に相当する.

表 13: 最良の手の分布とその成績 (平均収益最大のケース)

パラメータ	値
グーの出現率	0.00
チョキの出現率	0.63
パーの出現率	0.37
分布インデックス	38
勝利数	33,747
総収益	349,740
平均収益	3.4974

4.4.2 相手の戦略が常に変化する場合 (機械学習)

本実験では, セクション 3.6 で述べた仕様に基づいて, 簡易的な機械学習モデルを実装した. セクション 4.4.1 と同様に, 勝率比の値を 0.01 刻みで全て検証する形式で実験を行った. 各比率に対して, シミュレーションは 100 000 回繰り返し実施され, その都度勝利数, 獲得報酬の合計, そして平均報酬を算出した.

各比率ごとのシミュレーションが完了した後に得られたデータを基に, 比率ごとの比較を行ったグラフを作成した. 結果は図 13 に示す.

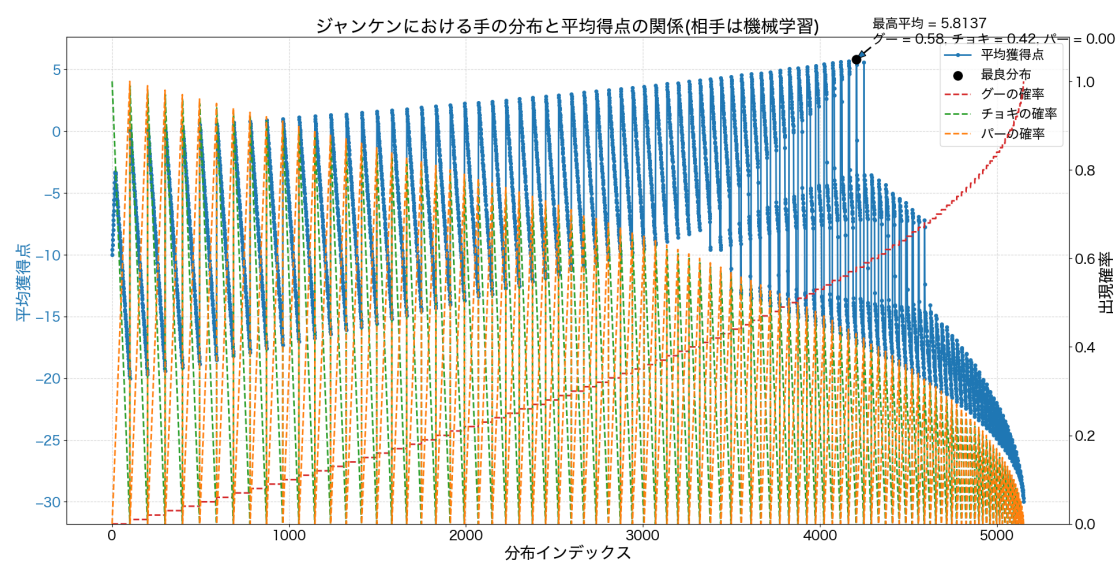


図 13: 手の出し比率による比較 (機械学習による対戦相手)

相手が動的に戦略を変化させる場合における最適な手の比率は、表 14 に示されている。

表 14: 相手が積極的に学習する戦略を採用していると仮定した場合の最適なハンド比率

手の種類	出現率 (%)
グー	58
パー	0
チョキ	42

シミュレーションプログラムの出力結果である表 15 を参照すると、表 14 に示された手の比率を使用することで、平均収益は 5.8137 となり、総収益は 581370 であることがわかる。100000 回のシミュレーションのうち、勝利した回数は 58198 回であり、総反復回数に対して 58% の勝率であった。

表 15: 最良の手の比率とその成績（パーを出さない戦略）

パラメータ	値
グーの出現率	0.58
チョキの出現率	0.42
パーの出現率	0.00
分布インデックス	4206
勝利数	58,198
総収益	581,370
平均収益	5.8137

4.5 課題 5: ビンゴのシミュレーション

本シミュレーションでは、先に節 2.1.5 で説明した通り、3 つの異なる条件を設定して検証を行った。75 個の数字から選択する 5×5 のビンゴカードの組み合わせが非常に多いため、これまでのシミュレーションと比較して、出現率の精度を確保するには圧倒的に多くの繰り返し回数が必要となる。それぞれの条件において、1000 万回の繰り返しを実施した。

4.5.1 7回以内で当たる

7回の数字コール内でビンゴ（任意の1列の完成）を達成する条件でビンゴシミュレーションを実行した結果は、図14に示されている。

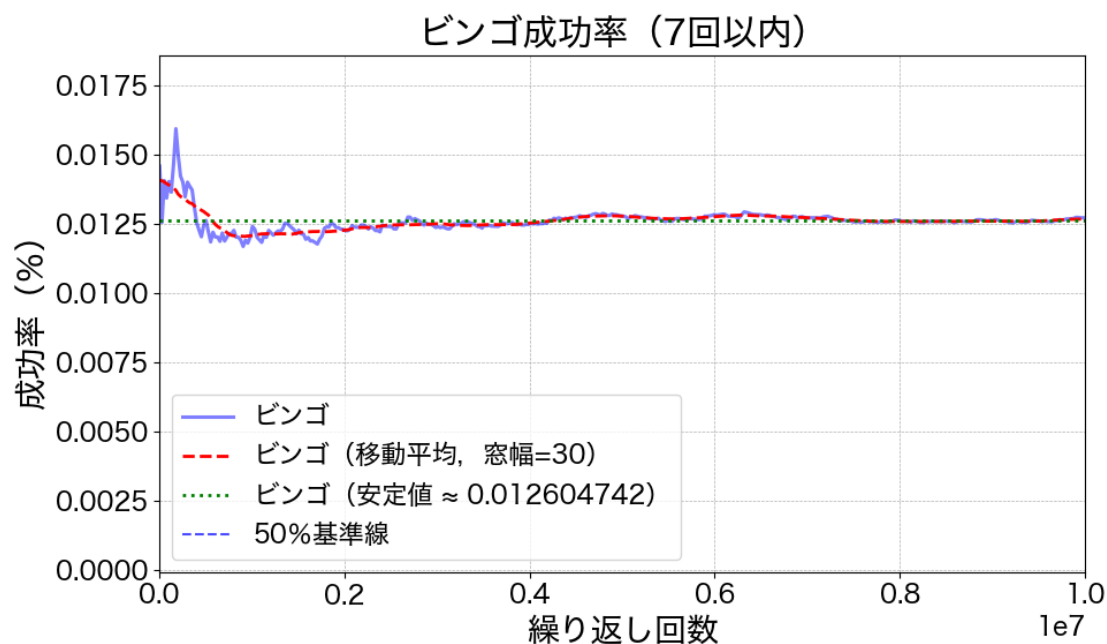


図 14: 7ラウンド以内にビンゴが出る確率 (1000 万回繰り返し)

200 万回の反復までは、グラフは不安定で大きな変動が見られた。しかし、400 万回に到達すると徐々に安定傾向が現れ始め、800 万回以降にはほぼ直線的な推移となり、安定した値に収束していることが確認できる。最終的に、データの最後の 100 行を平均した結果、シミュレートされた出現率は 0.012604742%であり、これはおよそ 7932 回に 1 回の出現頻度に相当する。

4.5.2 最短回数 (4回)

4回の数字コール内でビンゴ（任意の1列, 列, または斜めの完成）を達成する条件でビンゴシミュレーションを実行した結果は、図15に示されている。

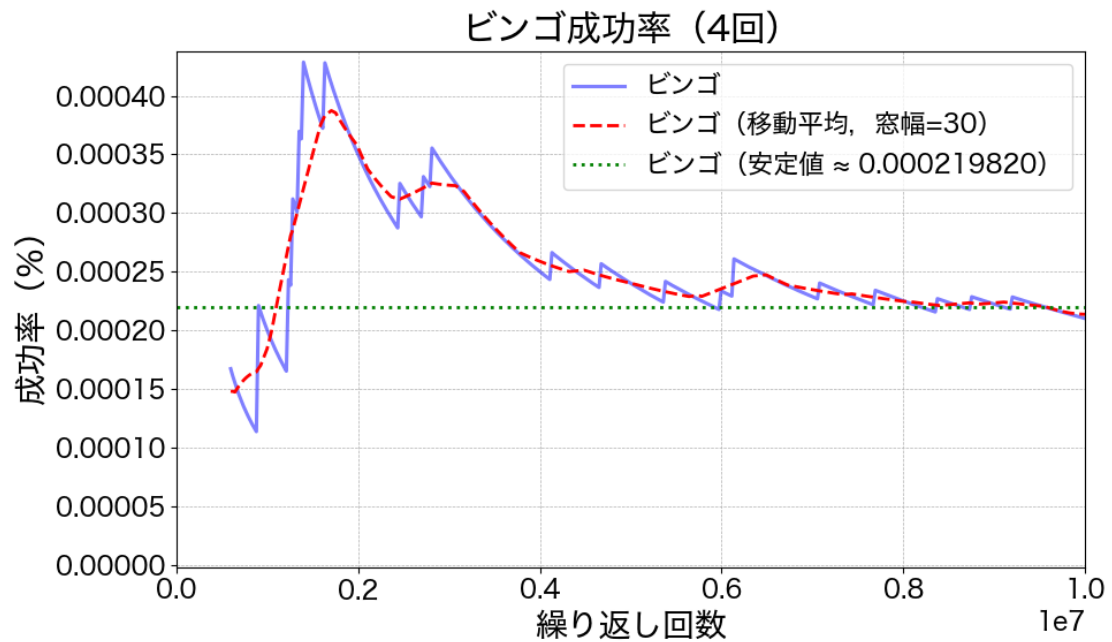


図 15: ちょうど 4 回目の数字コールでビンゴを達成する確率 (1 000 万回の繰り返し)

本シミュレーションにおいては, 4 百万回の繰り返し後も移動平均は非常に不安定であった. 移動平均が安定し始めたのは 6 百万回の繰り返しを超えてからであり, その後は図示した SMA (単純移動平均) に示されるように下降傾向を示した. 最後の 100 行のデータを平均すると, シミュレーションによる出現率は 0.000219820% (約 45 万 4,834 回に 1 回の発生) となる.

4.5.3 当たりとなる最も遅いゲーム回数 (71 回)

前節 2.1.5 で述べたように, ビンゴゲームにおいて完全な列ができることなく数字が呼ばれる最大回数は 70 回である. つまり, 71 回目の数字呼び出しでは必ず完全な列が達成されることになる. この条件でビンゴシミュレーションを実行した結果は, 図 16 に示されている.

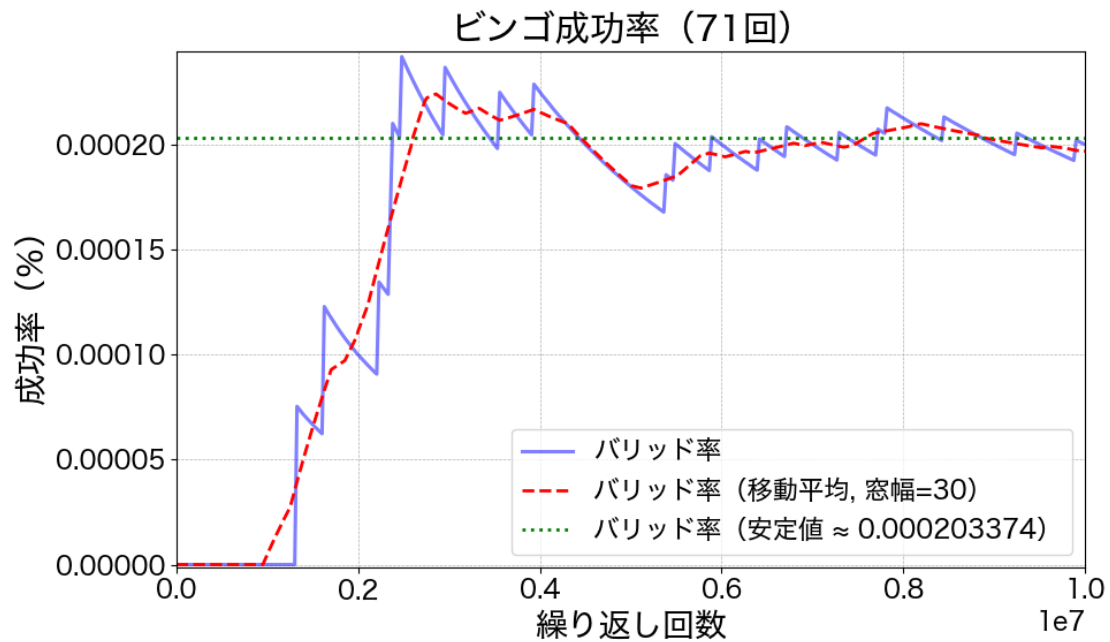


図 16: Rate of hitting bingo in 71 rounds.(10 million repetitions)

本シミュレーションにおいては、8 百万回の繰り返しの達するまでは移動平均が不安定であった。8 百万回を超えた後に移動平均は安定し始め、ランニングアベレージと単純移動平均（SMA）を比較することでその安定傾向が確認できる。シミュレーション結果の最後の 100 行の平均をとると、出現率は 0.000203374%（約 49 万 0,580 回に 1 回の発生）となる。

5 分析と理論

5.1 課題 1: 7 つのサイコロが同じ目を出すシミュレーション

式 2 で計算した 7 個のサイコロが単一の投擲で全て同じ目を出す理論的出現率と、表 7 のシミュレーション結果および図 8 の可視化結果を比較すると、シミュレーションの精度は高く、偏差は 3.58% にとどまり誤差範囲の 5% 内に収まっていることが確認できる。5 百万回の試行を実施したにもかかわらず、ランニングアベレージが理論値に完全には収束しなかったのは、7 個のサイコロが全て同じ目を出すという条件が極めて稀な現象であるためである。さらに、各サイコロの目は互いに独立かつランダムに生成されるため、この条件の発生頻度は約 $1/450$ に過ぎない。

5 百万回の試行回数は維持しつつ投擲するサイコロの数を変化させた結果を表 8 に示す。サイコロの数が増加するにつれて、ランニングアベレージが理論的出現率

に収束しない傾向が強くなることが分かる。サイコロの数が増えるほど、期待条件の発生確率は低下するためである。例えば、サイコロの数を 10 個に増やすと、理論的出現率は 0.000009923% となり、約 1 008 1785 回の試行に 1 回しか発生しない極めて稀な事象となる。これは 5 百万回の試行では、10 個のサイコロが全て同じ目を示す条件が発生しない可能性が高いことを意味する。したがって、条件が稀であるほど、最も正確なシミュレーション出現率を得るためにはより多くの試行回数が必要であることを示している。

5.2 課題 2: 半径 7 の円の面積

本シミュレーションでは、円の外周と二次元空間の端に隙間がないものと仮定する。半径を 7 とした場合、二次元空間の面積は $2 \times 7 \times 2 \times 7 = 196$ となる。円の面積を求め、それを空間の面積で割ることで、式 4 に示すように、円の面積が全体空間に占める割合は約 0.785、すなわち 78.5% となる。 x 座標および y 座標を独立に生成したランダムな点を 4000 個プロットすることで、最終的なシミュレーション結果として 78.85% の割合が得られ、理論値からの偏差は 0.3949% であった。

円の面積が二次元空間全体の 78.5% を占めているため、本シミュレーションでは比較的少ない繰り返し (500 回) でも平均値がある程度理論値に近づいた。実際、500 点で得られた偏差は 2.88% と、許容誤差の 5% 以内に収まっていた。しかし、図 9 に示されるように、500 回時点では移動平均は依然として不安定であり、十分な収束が得られていないことが分かる。モンテカルロ法において、より正確な出現率を得るためには十分な繰り返し回数が必要である。グラフからは、少なくとも 2500 回の繰り返しで初めて移動平均が安定し、徐々に理論値に収束していく様子が確認できる。

また、円の半径を変更しても結果は変わらない。式 4 の理論比率を、式 15 に示すように展開すると、円の半径が分子および分母に共通して現れるため打ち消し合い、最終的な比率は $\frac{\pi}{4}$ に収束する。このことから、半径の大きさに関わらず、円と正方形の面積比率は一定であることが分かる。

$$A = 2r \times 2r = 4r^2 P = \frac{\text{円の面積}}{\text{領域の面積}} = \frac{\pi r^2}{A} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4} \quad (15)$$

したがって、半径を変更した場合のシミュレーション結果を表 10 で観察すると、すべての半径において同一の理論値に収束していることが確認できる。しかしながら、4000 回の繰り返し後には、通常 1% 未満の偏差を期待することができる。実際、半径 4 を用いたシミュレーションでは、偏差が 1.6% を超える突発的な上昇が確認

された。この偏差は5%の許容誤差範囲内ではあるが、理論値とのズレとしては小さく、データの信頼性や意味に大きな影響を与えるものではないと考えられる。

5.3 課題3: 異身長男女を一行に並べる

サイコロや面積のシミュレーションとは異なり、10人の並びに関するシミュレーションは手計算で解くには複雑すぎる。各人物が異なる身長を持つ10人の列では、考えられる順列は $10! = 3628800$ 通り存在し、これは式6に示す通りである。すべての順列に対して「各女性の左側に、より背の高い男性が1人以上存在する」という条件を手動で検証することはほぼ不可能である。そこで、コード1に示すPythonスクリプトにより全順列を総当たりで検証した結果、条件を満たす順列は893025通り、すなわち全体の24.61%であることが判明した。

この理論値を元に、10000回のシミュレーションを実施した。表11を参照すると、平均偏差は1.82%であり、5%の誤差範囲内で理論値24.61%に収束していることが確認できる。しかし、図11で確認すると、ランニングアベレージはおよそ2000回付近で安定し、その後8000回までの間に更なる収束は見られず、偏差は約1.3%で停滞している。このことから、繰り返し回数を増やすことによる収束の向上には限界があり、一定回数を超えると収穫逨減が発生することが分かる。

モンテカルロシミュレーションにおいては、正確な出現率を得ることが目的ではなく、出現率のおおよその傾向や一般化を得ることが目的である。あくまでシミュレーションであるため、理論値に完全に一致する結果を得ることは不可能である。しかし、偏差の小さい安定したランニングアベレージを得ることで、出現率の推定としては十分であるといえる。

5.4 課題4: ジャンケンの割合

前述の節2.1.4にて述べた通り、相手が完全にランダムでいずれの手にも偏りがないと仮定した場合、グーを選択することによる理論的な期待値は0.0であり、パーとチョキの割合を合計して1.0となる範囲で任意に設定できるとされる。また、式10において、理論的に期待できる平均利得は3.33であることが計算された。

すべての比率の組み合わせに対し、図12に示す。最良のシミュレーション結果は、グー0%、パー37%、チョキ63%という手の出し方であり、この比率により平均利得は3.4974となった。この結果は、グーを一切使用せず、パーとチョキのみで構

成される理論的な最適比率と整合している。

また、理論的な平均利得 3.33 に対し、シミュレーションによって得られた平均利得 3.4974 は、5.026% の偏差である。これは 5% 以内の誤差範囲に収まっており、シミュレーションとして十分に正確であると判断できる。

特に、パーで敗北した場合には相手に 30 ポイントを与え、自身が 30 ポイントを失うという最大損失が発生する。本シミュレーションでは、そのようなリスクを回避するため、グーによる勝利の期待値が低い点を考慮し、グーを一切使用しないことで損失の最小化が図られている。

本シミュレーションの結果は、相手が 6 つの入力値を用いた能動的な学習戦略を導入した場合、全く異なる傾向を示す。図 13 を分析すると、最良の手の出し方は、グー 58%、パー 0%、チョキ 42% という比率であり、この比率を用いた際の平均利得は 5.8137 となった。これは、偏りのない相手との対戦時に得られた平均利得に対して 66.23% の増加である。

偏りのない相手との対戦では損失の最小化を重視し、グーを避けるという戦略が最適であったが、学習型の相手との対戦では、むしろ勝利回数を最大化するために、グーとチョキのバランスを取りつつ積極的に手を出す比率へと戦略が大きく変化している。このことは、偏りのある相手に対する最適戦略が、偏りのない相手に対するそれとは全く逆転することを意味している。

ただし、今回使用した相手の機械学習モデルは、隠れ層を一切持たない非常に単純な構造である。今後、より多くの入力ベクトルや複数の隠れ層を導入することで、相手モデルの予測精度が向上し、我々の手をより正確に予測される可能性がある。その場合、利得の最大化は一層困難になり、本シミュレーションで得られた結果を上回することは難しくなると考えられる。

5.5 課題 5: ビンゴのシミュレーション

ビンゴシミュレーションでは、第 2.1.5 節で定義された 3 つの異なる条件が検証対象となった。条件の複雑さや目的とする事象の希少性から、各シミュレーションは 1,000 万回の反復を要した。7 ターン未満でビンゴを達成するという条件においては、図 14 に示すように、400 万回の反復以降にローリング平均が単純移動平均 (SMA) と一致し始め、安定化傾向を見せている。シミュレーションデータの最後の 30 行を平均した結果、出現率は 0.012604742% となり、これはおよそ 7932 回に 1 回の頻度に相当する。400 万回以降は、平均値の変動がほとんど見られず、結果が

収束していることが確認できる。

これは、第 4.3 節の実験 3 においても観測された現象であり、反復回数をさらに増加させたとしても、精度向上には限界がある「収穫逡減」の一例である。より単純なシミュレーションにおいては、計算資源を節約するために反復回数を適切に削減することが有効である。

また、ビンゴカード上の任意のラインでビンゴを達成するまでに要するコール数を記録する簡単なスクリプトを実行することで、図 17 に示すようなベルカーブとして可視化することが可能である。

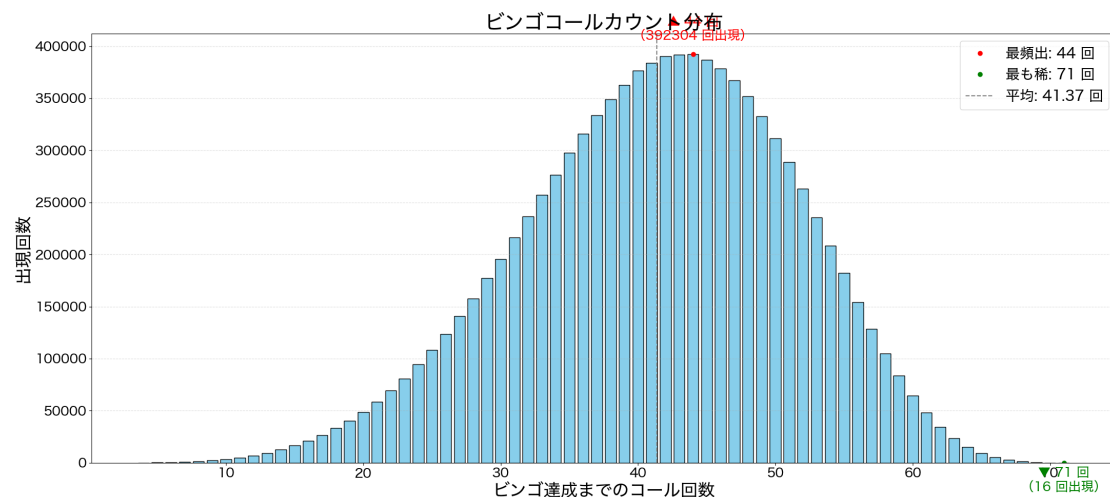


図 17: ビンゴコールカウント分布 (1 千万回)

図 17 に示されたこの分布曲線は、実験結果として期待される傾向をよく表している。最も一般的なビンゴ達成までのコール回数は 44 回であり、分布の両端には非常に稀なケースとして、4 回でビンゴが成立する場合や 71 回を要する場合が存在する。

特に、最小回数（4 回）でビンゴを達成するという条件で実行されたシミュレーションでは、出現率は 0.000219820% であった。この値はシミュレーション出力の最後の 30 行から平均を取って算出されたものである。図 15 に示されるローリング平均の収束を分析すると、600 万回の反復を超えたあたりで平均値が安定化し、最終的な出現率に向かって徐々に収束している様子が確認できる。

しかし、安定化以降の収束トレンドが十分に長期間観測されているとは言えず、本シナリオのような極めて稀な事象に対しては、1000 万回の反復では依然として正確性に疑問が残る可能性がある。よって、より精緻な結果を得るには更なる反復

回数の増加が必要とされるだろう。

図16に示されるように、1000万回の反復の結果、出現率の最終平均は約 $2.03374 \times 10^{-4}\%$ となった。このとき、71回ビンゴの平均値と4回ビンゴの平均値との間には、8.09%の偏差が見られた。

図17に示されるベル型分布の左右両端に位置する、極めて稀な事象であることを踏まえると妥当な結果である。また、本シミュレーションにおける分布曲線はやや右に偏っており（高コール回数側に重い）、そのため、71回ビンゴの出現率が4回ビンゴよりもわずかに高くなること（+8.09%）は自然な傾向であり、統計的にも十分許容される範囲である。

しかしながら、4回ビンゴのシミュレーションと同様に、今回の条件の出現率が極めて低く（約 $1/491596$ ）なっているため、ローリング平均のグラフは安定化した後に十分な収束時間を得ることができなかった。すなわち、本シミュレーションもまた反復回数が不十分であり、得られた最終的な平均値が十分な精度を持つとは限らない。

6 まとめ

モンテカルロシミュレーションは、解析的な解法が実用的でない、あるいは不可能なシステムにおいて確率を推定するための不可欠な手法である。特に、全体の順列数が膨大である場合や、システムの挙動が極めて複雑もしくは適応的で直接的なモデル化が困難な場合に、その有効性が発揮される。

現代のコンピュータは、ある程度のシナリオを総当たりで探索できる性能を有しているが、モンテカルロ法はそれに比べて計算コストが格段に低く、かつスケラブルに統計的に信頼性の高い結果を得ることが可能である。ただし、その精度は繰り返し回数および乱数生成器の質に大きく依存する。特に発生頻度の低い事象やエッジケースにおいては、意味のある収束を得るために数百万回以上の試行が必要となる場合がある。

また、対戦的要素や適応的な振る舞いを含むような動的環境においては、解析的モデリングは信頼性を欠くことが多く、モンテカルロ法のようなシミュレーション手法が不可欠となる。この手法により、静的な数式では表現不可能な、時間と共に変動する確率分布の推定が可能となる。

シミュレーション中の収束性や安定性を評価するためには、移動平均やトレンド分析といったモニタリング手法が重要である。これらの評価を行わない場合、結果はノイズや偶然性に大きく影響され、誤った結論を導く可能性がある。

さらに、乱数生成の品質はシミュレーション精度に直接影響を与えるため、決して軽視すべきではない。擬似乱数生成器は本質的に完全なランダム性を持たず、長期的に見ると予測可能なパターンを生じさせることがある。したがって、統計的バイアスを最小限に抑え、均一性を確保するためには、高品質な乱数生成器の使用が不可欠である。

以上より、モンテカルロシミュレーションは、複雑なシステムにおける確率的理解を深めるための重要なツールであり、適切な設計と十分なスケール、および厳密な評価を伴って初めて、その真価を発揮する。

7 発表についての感想

7.1 齊藤 隼

本発表は、各数値がどのように導かれたかについての詳細な説明とともに、非常に有用なデータを提示しており、対象となるテーマに対する深い理解が伺えました。結果の重要性も明確に伝わっており、分析自体は非常に評価に値します。

一方で、スライドの構成についてはやや素朴であり、グラフや表に対する説明が不足している点が気になりました。特に、図表にキャプションや注釈がないため、聴衆がデータの意味をスライドだけで正確に把握するのが難しく、発表者の口頭説明がなければ理解が困難な場面も見受けられました。また、既定のテンプレートをそのまま使用しているため、視覚的な訴求力に欠け、内容の印象がやや薄れてしまっています。

とはいえ、スライドのレイアウトを工夫し、図表に簡潔な説明やラベルを付加するなど、視覚的かつ説明的な強化を図ることで、発表の分かりやすさと説得力は大きく向上すると考えられます。全体としては構成がしっかりしており、内容の質も高いため、今後さらに伝わりやすい資料作成を意識すれば、より優れた発表になるでしょう。

7.2 神力 奏太

前回の発表とは対照的に、神力のスライドは美しく、視覚的にわかりやすく、見ていて楽しいものでした。しかし、プレゼンテーションの話し方や説明のスタイルに

はやや物足りなさがあり, 各データがどのようにして得られたのか, あるいはなぜそのような結果になったのかといった背景情報が不足していました. もう少し情報を補足し, 実験内容に関しても要素を変えるなどの工夫を加えることで, 聴衆にとって説明されている概念がより理解しやすくなると思います.

8 付録

本実験に利用したソースコードなどは GitHub にホストされている. <https://github.com/SorataBaka/kosen-5/tree/main/random-simulation>

参考文献

- [1] Wolfgang Hörmann and Gerhard Derflinger. A portable uniform random number generator well suited for the rejection method. WorkingPaper 3, Institut für Statistik und Mathematik, Abt. f. Angewandte Statistik u. Datenverarbeitung, WU Vienna University of Economics and Business, 1992.
- [2] IBM. What is monte carlo simulation?, 2021. Accessed: 2025-07-20.
- [3] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Boston, MA, 3rd edition, 1998.
- [4] Daniel Lemire. Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 29(1):1–12, 2019.
- [5] Marcin Majka. Moving averages in time series analysis. 09 2024.
- [6] Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, January 2019.
- [7] OpenBSD Project and BSD Manual Pages. *arc4random(3): High-quality CSPRNG functions*, 2025. Manual page, OpenBSD (and FreeBSD/macOS).
- [8] Giovanni Ossola and Alan D. Sokal. Systematic errors due to linear congruential random-number generators with the swendsen-wang algorithm: A warning. *Physical Review E*, 70(2), August 2004.
- [9] The OpenBSD Project. arc4random — cryptographic random number generator. <https://man.openbsd.org/arc4random>, 2024. Accessed: 2025-07-20.